

## Question 1: Banker's Algorithm

The Safety Algorithm searches for a safe sequence by repeatedly finding a process whose remaining Need can be satisfied by the current Work vector. The Resource Request Algorithm first checks the request against Need and Available, temporarily grants it, reruns the Safety Algorithm, and commits the request only if the resulting state remains safe.

Sample scenario:

Allocation

P0: 0 1 0

P1: 2 0 0

P2: 3 0 2

P3: 2 1 1

P4: 0 0 2

Max

P0: 7 5 3

P1: 3 2 2

P2: 9 0 2

P3: 2 2 2

P4: 4 3 3

Available: 3 3 2

The initial safe sequence is  $P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$ . Two request tests are included: a safe request  $P0 = (0,0,1)$ , and an unsafe request  $P0 = (0,0,2)$ , which is rejected because granting it leaves the system unsafe.

### Sample output log

Processes: 5

Resource types: 3

Available: 3 3 2

Initial safety check:

Safe sequence:  $P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$

System is in a SAFE state.

- Safe request scenario -

Request from P0: 0 0 1

Safe sequence:  $P3 \rightarrow P4 \rightarrow P1 \rightarrow P2 \rightarrow P0$

Request granted - resulting state is safe.

-Unsafe request scenario -

Request from P0: 0 0 2

Request denied - would lead to unsafe state.

This log demonstrates the required initial safe state, a request that is safely granted after a safety check, and a request that is rolled back because it would produce an unsafe state.

## Question 2: Deadlock Detection using Resource Allocation Graph

The program represents resource ownership using an allocation matrix and waiting relationships using a request matrix. A wait-for edge  $P_i \rightarrow P_j$  is created when  $P_i$  requests a resource currently allocated to  $P_j$ . A DFS-based cycle detector then checks the resulting wait-for graph. One scenario is acyclic and one contains a three-process circular wait.

### Sample output log

Deadlock Detection using Wait-for Graph

- Scenario 1: No deadlock -

Wait-for graph edges:

P0 -> P1

P1 -> P2

P2 -> P3

No deadlock detected.

- Scenario 2: Deadlock -

Wait-for graph edges:

P0 -> P1

P1 -> P2

P2 -> P0

Deadlock detected.

Deadlock cycle: P0 -> P1 -> P2 -> P0

## Question 3: Deadlock Prevention via Resource Ordering

Two xv6 semaphore-protected locks are used. The BAD version intentionally acquires Lock1 → Lock2 in Process A and Lock2 → Lock1 in Process B, creating circular wait. The FIXED version makes both processes acquire Lock1 before Lock2. Because every process follows the same global order, circular wait cannot form.

### Observed BAD-version output

Resource Ordering - BAD version

This version intentionally creates a deadlock.

Parent: waiting for processes...

Expected deadlock:

Process A holds Lock1 and waits for Lock2.

Process B holds Lock2 and waits for Lock1.

Interrupt this run manually after observing the deadlock.

Process A: starting

Process A: trying to acquire Lock1

Process A: acquired Lock1

Process B: starting

Process B: trying to acquire Lock2

Process B: acquired Lock2

Process B: waiting for Lock1

Process A: waiting for Lock2

The final two lines demonstrate the circular wait: A owns Lock1 and waits for Lock2, while B owns Lock2 and waits for Lock1. The run is intentionally interrupted because it is designed to remain blocked.

### Observed FIXED-version output

Resource Ordering - FIXED version

Both processes acquire locks in the order:

Lock1 -> Lock2

Process A: starting

Process A: trying to acquire Lock1

Process A: acquired Lock1

Process A: trying to acquire Lock2

Process A: acquired Lock2

Process A: critical section

Process A: released Lock2 and Lock1

Process A: completed successfully

Process B: starting

Process B: trying to acquire Lock1

Process B: acquired Lock1

Process B: trying to acquire Lock2

Process B: acquired Lock2

Process B: critical section

Process B: released Lock2 and Lock1

Process B: completed successfully

Both processes completed successfully.

No deadlock occurred.

The fixed run completes because both processes request the locks in the same order. A process may wait for Lock1, but no process can hold Lock2 while waiting for Lock1, so the circular-wait condition is removed.

## Question 4: Combined Synchronization & Deadlock Avoidance

This program extends the Dining Philosophers idea to five real xv6 processes and three resource pools: two Printers, one Scanner, and two Disks. Each process requires two resource types. Counting semaphores represent the available instances. Resource ordering is used as the deadlock-prevention strategy: Printer = 0, Scanner = 1, Disk = 2, and every process acquires its two resources in increasing order.

Process resource requirements:

P0 -> Printer + Scanner  
P1 -> Scanner + Disk  
P2 -> Printer + Disk  
P3 -> Printer + Scanner  
P4 -> Scanner + Disk

Resources are released in reverse order after the simulated work completes. This maintains correct synchronization and prevents circular wait. The following is the observed successful output pattern from the xv6 run; all five processes completed all five cycles.

### Observed output log

Resources: 2 Printers, 1 Scanner, 2 Disks  
Strategy: Resource Ordering  
Global order: Printer -> Scanner -> Disk  
Each process requires 2 resources  
Each process will perform 5 cycles

Process 0: starting cycle 1  
Process 0: requesting Printer  
Process 0: granted Printer  
Process 0: requesting Scanner  
Process 0: granted Scanner  
Process 0: both resources acquired  
Process 0: starting work  
Process 0: work completed  
Process 0: released Scanner  
Process 0: released Printer  
Process 0: completed cycle 1  
...  
Process 0: ALL CYCLES COMPLETED

Process 1: ALL CYCLES COMPLETED  
Process 2: ALL CYCLES COMPLETED  
Process 3: ALL CYCLES COMPLETED  
Process 4: ALL CYCLES COMPLETED

All 5 processes completed successfully.  
No deadlock occurred.  
All resources were shared within their limits.